FastAPI Interview Guide

Detailed FastAPI notes for API development, deployment, and RAG serving

Detailed interview preparation and practice notes. Text only. No tables or images.

Use this PDF as source data for your RAG chatbot and as revision material before interviews.

1. FastAPI introduction and when to use it

Concept

FastAPI is a modern web framework for building APIs in Python. It is used for backend services, ML model serving, RAG endpoints, internal tools, and microservices.

Important details

It provides automatic request validation, type-hint based development, OpenAPI docs, support for async, dependency injection, middleware, and easy testing.

Common mistakes

Do not say FastAPI itself makes CPU-heavy ML models faster. It improves API development; performance also depends on server, model loading, and I/O handling.

Interview answer style

Say FastAPI is built for high-performance APIs with automatic docs and Pydantic validation, useful for exposing Python logic as HTTP endpoints.

Small code example

```
from fastapi import FastAPI
app = FastAPI()
```

Interview and practice focus

Be ready to explain fastapi introduction and when to use it in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

2. FastAPI project structure

Concept

A clean FastAPI project separates routes, schemas, services, database code, and configuration. This prevents main.py from becoming too large.

Important details

Common folders include routers, schemas, services, core/config, models, and tests. For a RAG API, keep rag_pipeline.py separate from the API route file.

Common mistakes

Do not put all logic directly inside endpoint functions. Do not hardcode secrets. Do not mix UI code and API code unnecessarily.

Interview answer style

Explain that API layer receives requests, service layer handles logic, and response model structures the output.

Small code example

```
app/
  main.py
  routers/chat.py
  schemas.py
  services/rag_service.py
```

Interview and practice focus

Be ready to explain fastapi project structure in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

3. Running FastAPI with Uvicorn

Concept

FastAPI applications are commonly served using Uvicorn, an ASGI server. During development, --reload restarts the server when code changes.

Important details

In production, use a proper deployment command with host 0.0.0.0 and the platform port. The module path format is file_name:app_object.

Common mistakes

Do not use --reload in production. Do not forget to expose the correct port on deployment platforms.

Interview answer style

Interview answer: Uvicorn runs the ASGI application and handles HTTP requests to the FastAPI app.

Small code example

```
uvicorn main:app --reload
uvicorn main:app --host 0.0.0.0 --port 8000
```

Interview and practice focus

Be ready to explain running fastapi with uvicorn in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

4. GET, POST, PUT, PATCH, DELETE

Concept

HTTP methods describe the action. GET reads data, POST creates or processes data, PUT replaces data, PATCH partially updates data, and DELETE removes data.

Important details

For a chatbot endpoint, POST is usually better because the question is sent in the request body and may be long. GET is good for health checks.

Common mistakes

Do not send large or sensitive data in query strings. Do not use GET for operations that change server state.

Interview answer style

Explain REST basics and choose POST for /ask because it accepts a question body.

Small code example

```
@app.get("/health")
def health(): return {"status":"ok"}
```

Interview and practice focus

Be ready to explain get, post, put, patch, delete in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

5. Path parameters

Concept

Path parameters are dynamic values inside the URL path. FastAPI validates them using type hints.

Important details

Use path parameters for resource identifiers like user_id, item_id, or document_id. FastAPI returns validation errors for wrong types automatically.

Common mistakes

Do not use path parameters for optional filters; use query parameters for filters and pagination.

Interview answer style

Say path params identify a specific resource and are declared in the route path.

Small code example

```
@app.get("/documents/{doc_id}")
def get_doc(doc_id: int): return {"doc_id": doc_id}
```

Interview and practice focus

Be ready to explain path parameters in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

6. Query parameters

Concept

Query parameters are optional or filtering values after ? in a URL. They are good for pagination, search filters, limits, and sorting.

Important details

Default values make parameters optional. Without a default, FastAPI treats them as required unless Optional is used.

Common mistakes

Do not confuse path and query parameters. Do not accept unbounded limit values in production.

Interview answer style

Mention skip and limit as common examples.

Small code example

```
@app.get("/items")
def items(skip: int = 0, limit: int = 10): return []
```

Interview and practice focus

Be ready to explain query parameters in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

7. Request body and Pydantic models

Concept

Request bodies carry structured input data. Pydantic models define the schema and validation rules.

Important details

FastAPI reads the body, validates it, converts types where possible, and gives the endpoint a model object. This reduces manual if checks.

Common mistakes

Do not accept raw dict everywhere unless the schema is truly dynamic. Typed models improve docs and safety.

Interview answer style

Say Pydantic models make APIs self-documenting and validate input automatically.

Small code example

```
from pydantic import BaseModel
class Question(BaseModel):
    text: str
```

Interview and practice focus

Be ready to explain request body and pydantic models in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

8. Response models

Concept

Response models define what the API returns. They improve consistency, documentation, and data hiding.

Important details

Use response_model to ensure only expected fields are returned. This is useful when internal objects contain sensitive or unnecessary fields.

Common mistakes

Do not return database objects directly without shaping responses.

Interview answer style

Explain that response models validate output and document the API response.

Small code example

```
class Answer(BaseModel):
    answer: str
@app.post("/ask", response_model=Answer)
```

Interview and practice focus

Be ready to explain response models in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

9. Validation with Field

Concept

Pydantic Field adds constraints and metadata such as min_length, max_length, examples, and descriptions.

Important details

This is useful for validating questions, user input, page limits, and configuration. Better validation means fewer runtime errors.

Common mistakes

Do not rely only on frontend validation; APIs must validate on backend too.

Interview answer style

Interview answer: validation ensures input follows expected shape before business logic runs.

Small code example

```
from pydantic import Field
text: str = Field(min_length=1, max_length=1000)
```

Interview and practice focus

Be ready to explain validation with field in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

10. HTTPException and status codes

Concept

HTTPException returns controlled error responses. Status codes communicate the result: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, and 500 Server Error.

Important details

Use specific status codes. Avoid exposing internal exception messages to users in production.

Common mistakes

Do not return 200 for errors. Do not use 500 for validation problems caused by user input.

Interview answer style

Explain that errors should be clear, consistent, and safe.

Small code example

```
from fastapi import HTTPException
raise HTTPException(status_code=404, detail="Not found")
```

Interview and practice focus

Be ready to explain httpexception and status codes in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

11. Dependency injection with Depends

Concept

Depends lets you reuse setup logic across endpoints. Dependencies can provide config, database sessions, authentication results, or shared services.

Important details

It keeps endpoint functions clean and testable. Dependencies can also be overridden during testing.

Common mistakes

Do not create expensive objects inside dependencies for every request if they can be reused.

Interview answer style

Say dependency injection helps manage shared resources and cross-cutting concerns.

Small code example

```
from fastapi import Depends
def get_service(): return service
```

Interview and practice focus

Be ready to explain dependency injection with depends in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

12. Application startup and shared resources

Concept

Expensive resources such as ML models, embedding models, database connections, and vector databases should be loaded once at startup or cached.

Important details

This prevents loading the same model or vector database for every request. For RAG, load ChromaDB and embedding model once, then reuse during questions.

Common mistakes

A common mistake is creating vector embeddings inside every /ask request. That makes the API slow and costly.

Interview answer style

Interview answer: load heavy resources once and reuse them across requests for better performance.

Small code example

```
rag = None
@app.on_event("startup")
def startup():
    global rag
    rag = RagPipeLine()
```

Interview and practice focus

Be ready to explain application startup and shared resources in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

13. Async endpoints

Concept

async def endpoints allow non-blocking I/O when using async libraries. They are useful for network calls and async database operations.

Important details

If you call blocking CPU-heavy code inside async endpoints, it can still block the event loop. Use correct libraries and architecture.

Common mistakes

Do not make everything async without understanding. File I/O, CPU-heavy ML, and blocking SDKs may still block.

Interview answer style

Explain async is best for I/O-bound work, not a magic speed boost for every task.

Small code example

```
@app.get("/async")
async def async_route():
    return {"ok": True}
```

Interview and practice focus

Be ready to explain async endpoints in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

14. Routers

Concept

APIRouter splits endpoints into modules. This keeps large applications organized.

Important details

Use prefixes like /chat, /users, /documents. Include routers in main.py. This helps maintainability and teamwork.

Common mistakes

Do not keep hundreds of endpoints inside main.py.

Interview answer style

Say routers are like mini FastAPI apps that group related endpoints.

Small code example

```
router = APIRouter(prefix="/chat")
app.include_router(router)
```

Interview and practice focus

Be ready to explain routers in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

15. Middleware and CORS

Concept

Middleware runs around requests and responses. CORS middleware allows browser frontends from specific origins to call your API.

Important details

Configure allowed origins carefully. During development * may be acceptable, but production should restrict origins.

Common mistakes

Do not use allow_origins=["*"] with credentials in production.

Interview answer style

Explain CORS is a browser security rule for cross-origin API calls.

Small code example

```
app.add_middleware(CORSMiddleware, allow_origins=["*"])
```

Interview and practice focus

Be ready to explain middleware and cors in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

16. Authentication basics

Concept

APIs can use API keys, bearer tokens, OAuth2, or session cookies. For simple projects, API key headers are easiest.

Important details

Secrets should be stored in environment variables. Authentication code can be placed in a dependency.

Common mistakes

Do not hardcode real tokens. Do not print secrets in logs.

Interview answer style

Interview answer: authentication verifies who is calling the API, authorization decides what they can access.

Small code example

```
API_KEY = os.getenv("API_KEY")
```

Interview and practice focus

Be ready to explain authentication basics in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

17. FastAPI and RAG API design

Concept

A RAG API usually has endpoints like /health, /ask, /documents, and maybe /reload-index. The /ask endpoint receives a question and returns answer plus optionally sources.

Important details

The pipeline should retrieve top-k chunks, build context, call the generation model, and return a grounded answer.

Common mistakes

Do not rebuild vector DB for each question. Do not expose raw confidential documents in API responses unless required.

Interview answer style

Explain a clean flow: request body -> validation -> retriever -> context -> generator -> response.

Small code example

```
@app.post("/ask")
def ask(q: Question):
    return {"answer": rag.ask_question(q.text)}
```

Interview and practice focus

Be ready to explain fastapi and rag api design in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

18. Testing FastAPI

Concept

TestClient allows testing FastAPI endpoints without manually running a server. Tests check status codes, response JSON, validation errors, and edge cases.

Important details

Keep business logic testable separately from endpoints. For RAG tests, mock the LLM to avoid external API dependency.

Common mistakes

Do not rely only on manual testing through Swagger UI.

Interview answer style

Interview answer: automated tests make API changes safer and catch regressions.

Small code example

```
client = TestClient(app)
res = client.get("/health")
assert res.status_code == 200
```

Interview and practice focus

Be ready to explain testing fastapi in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

19. Swagger UI and OpenAPI docs

Concept

FastAPI automatically generates interactive documentation at /docs and OpenAPI schema at /openapi.json.

Important details

This helps developers test endpoints, understand request bodies, and view response models. It is generated from type hints and Pydantic models.

Common mistakes

Do not expose internal-only docs publicly if the API is sensitive.

Interview answer style

Mention automatic docs as one of FastAPI biggest advantages.

Small code example

```
Open browser: /docs
```

Interview and practice focus

Be ready to explain swagger ui and openapi docs in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

20. Deployment best practices

Concept

A deployed FastAPI app should have one start command, environment variables configured, dependencies pinned, logs enabled, and heavy resources loaded once.

Important details

Use platform-specific port variables when needed. Add health check route. Avoid relying on local files not included in deployment.

Common mistakes

Do not use development reload mode in production.

Interview answer style

Interview answer: deployment needs reproducibility, secrets management, stable start command, and monitoring/logs.

Small code example

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

Interview and practice focus

Be ready to explain deployment best practices in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.

21. FastAPI interview practice questions

Concept

Common questions include: Why FastAPI? What is Pydantic? Difference between path and query params? What is Depends? How do you handle errors? How do you deploy? How do you serve ML models?

Important details

Practice by creating a /health endpoint, a POST /ask endpoint, validation model, custom HTTPException, and a simple dependency.

Common mistakes

Do not just memorize definitions; write a tiny API and explain every line.

Interview answer style

A strong candidate can connect FastAPI concepts with practical projects such as RAG chatbot deployment.

Small code example

```
print("Practice FastAPI daily")
```

Interview and practice focus

Be ready to explain fastapi interview practice questions in simple words, mention one real project use case, write a tiny example without looking, and identify one common error. Practice by creating a short program that uses this concept and then explain the output line by line.